Software Requirements Specification

for

Tech Company Web Crawler and Job Openings Tracker

Version 1.0 approved

Prepared by <author>

<organization>

<date created>

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ..... **ii**

1. Introduction ..... **i**
1.1 Purpose
1.2. Intended Audience and Reading Suggestions i
1.3 Product Scope i
1.4 References 2
2. Overall Description ..... **ii**
2.1 Product Perspective ii
2.2 Product Functions ii
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 5
2.7 Assumptions and Dependencies 5
3. External Interface Requirements **6**
3.1 User Interfaces 6
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 6
4. System Feature **7**
4.1 Job Crawling and Data Ingestion 7
4.2 Job Search and Viewing 7
4.3 Interview Practice (AI-Assisted) 8
4.4 Skill Gap Analysis (Resume vs Job) 9

5. Other Nonfunctional Requirements **9**
5.1 Performance Requirements 9
5.2 Safety Requirements 9
5.3 Security Requirements 10
5.4 Software Quality Attributes 10
5.5 Business Rules 10

Appendix A: Glossary **10**
Appendix B: Analysis Models **11**
Appendix C: To Be Determined List **11**

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the **Tech Company Web Crawler and Job Openings Tracker** system. The system automates the discovery, extraction, processing, and tracking of job postings from multiple sources (e.g., company career pages and job platforms) and provides a searchable, filterable view of consolidated job-opening data.

This document covers the complete system including the crawler, processing modules, backend services, data storage, and the user-facing interface.

1.2 Intended Audience and Reading Suggestions

Intended Audience

- **Developers/Maintainers:** Implement and maintain the crawler, processing pipeline, API, database layer, and UI.
- **Project Stakeholders/Product Owner:** Validate product scope, features, priorities, and acceptance expectations.
- **Testers/QA:** Derive test scenarios, verification criteria, and regression coverage.
- **Technical Writers (Optional):** Prepare user guides, deployment notes, and system documentation.

Reading Suggestions

- Read **Section 1** to understand objectives, scope, and references.
- Read **Section 2** to understand the system context, users, and constraints.
- Use **Sections 3–5** for detailed interface/feature requirements and nonfunctional requirements.

1.3 Product Scope

The product collects job postings from selected sources, normalizes and enriches them (e.g., cleaning text and extracting key fields such as title, company, location, skills, and posting URL), stores them in a database, and exposes them through an interface for tracking, searching, and filtering.

Goals and benefits

- Reduce manual effort required to search and monitor job openings.
- Provide a centralized, duplicated, and frequently updated job database.
- Enable filtering/search and basic insights (e.g., job trends by company, role, or location).

1.4 References

1) IEEE SRS guidance / IEEE-style SRS template ("srs_template-ieee").
2) Project source code repository (local): Tech Company Web Crawler and Job Openings Tracker.
3) Target sources' terms of use and robot.txt files (as applicable), and any public API documentation for job sources.

2. Overall Description

2.1 Product Perspective

The **Tech Company Web Crawler and Job Openings Tracker** is a **new, self-contained system** developed to replace the manual and fragmented process of finding and tracking job openings across multiple sources. Rather than relying on users to repeatedly visit different company career pages or job platforms, the product centralizes job discovery and tracking into one system.

This product is composed of multiple cooperating subsystems:

- **Crawler Subsystem (Data Collection):** Connects to external job sources (e.g., company career pages and job platforms) and extracts job posting content.
- **Processing/Enrichment Subsystem (Data Preparation):** Cleans, normalizes, and reduplicate extracted job postings; optionally enriches data using NLP/AI (e.g., skill extraction, keyword tagging, and summaries).
- **Backend/API Subsystem (Service Layer):** Provides programmatic access to stored job data for search, filtering, and retrieval; exposes endpoints that the UI and other clients can consume.
- **Data Storage Subsystem (Database):** Stores job postings, company/source metadata, crawl history, and processing outputs.
- **User Interface Subsystem (Presentation Layer):** A web interface that allows users to search/filter job openings and view detailed job information.

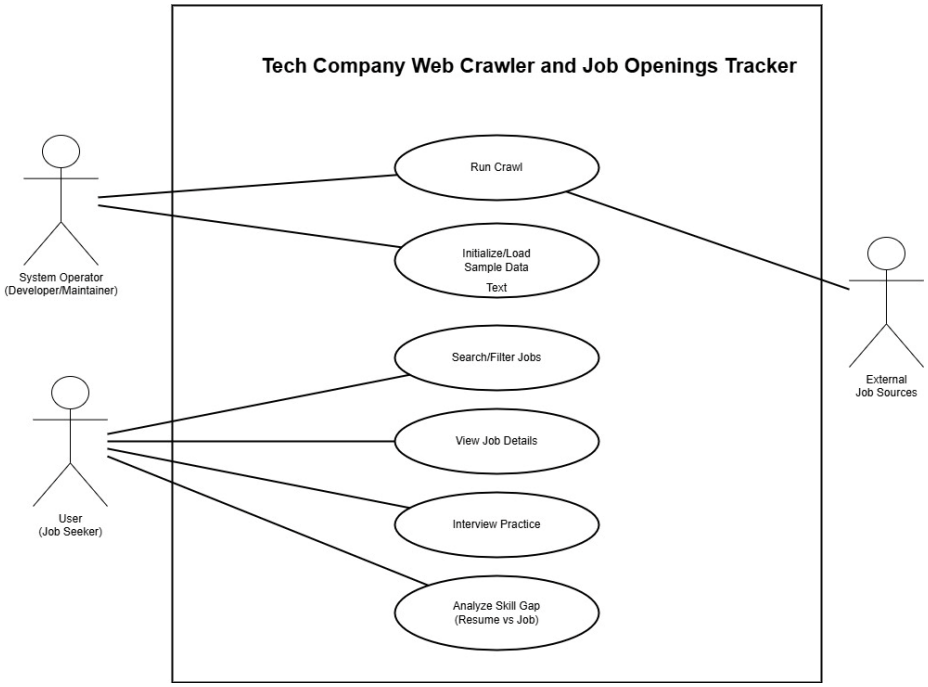
External Interfaces

- The system interfaces with **external websites** to retrieve job postings (HTTP/HTTPS).
- The UI interfaces with the backend through **API calls** (typically JSON over HTTP/HTTPS).
- The backend interfaces with the **database** to read/write job and metadata records.

2.2 Product Functions

The **Tech Company Web Crawler and Job Openings Tracker** enables administrators to configure job sources and initiate or schedule crawls to collect job postings from those sources. The system extracts key job information (such as title, company, location, description, and source URL), normalizes and stores it in a database, and attempts to reduce duplicates. Users can then search, filter, and view consolidated job postings

through a user interface or API. The system maintains basic crawl status and logs to support monitoring, troubleshooting, and testing.



Use Case Diagram or user stories to visualize how different users interact with the system

2.3 User Classes and Characteristics

The **Tech Company Web Crawler and Job Openings Tracker** is expected to be used by multiple user classes with different goals, responsibilities, and levels of technical expertise. The most important users to satisfy are **Job Seekers** (primary consumers of job listings and interview features) and **System Operators** (who ensure the crawler and database remain functional).

User (Job Seeker)

Job seekers are the primary users of the system. They use the product to search, filter, and review job postings collected from multiple sources, and they may use supporting features such as interview practice and skill-gap analysis. Job seekers are expected to have low to moderate technical skills and will interact with the system regularly (daily or weekly), especially during active job searching. Their main expectations are that results are easy to browse, relevant, and up to date.

System Operator (Developer/Maintainer)

System operators are responsible for keeping the system running and the job data current. They perform tasks

such as initializing or resetting data, triggering or scheduling crawls, monitoring crawl status/logs, and troubleshooting crawler failures due to source changes or access restrictions. System operators are expected to have high technical expertise and will use the system as needed, especially when onboarding sources, debugging extraction issues, or maintaining deployments. Although the system may not implement a formal "admin login" role, this user class represents the people who operate the system through technical access (e.g., API calls, server access, or internal tools).

External Job Sources (Third-Party Websites/Platforms)
External job sources are not human users but are important "system actors" that the crawler interacts with. These sources provide job posting pages that may vary in structure and availability over time. The system must interact with these sources in a controlled and respectful way (e.g., rate limiting and handling failures). This actor class is less important to "satisfy" in a user-experience sense, but it strongly affects system reliability and data quality.

2.4 Operating Environment

The Tech Company Web Crawler and Job Openings Tracker will operate as a Python-based application that can run in a local development setup or on a hosted server environment. The backend services are expected to run on a modern operating system such as **Windows 10/11** (development) or **Linux** (deployment), with **Python 3.x** available. The system requires a stable internet connection to access external job sources during crawling and to serve API requests to clients.

The backend operates as a web service (REST API) and should run alongside common development and runtime components including a web server process (e.g., Uvicorn for FastAPI) and supporting Python libraries for HTTP requests and HTML parsing. The data layer supports a relational database environment; in this project, the database may be configured as **SQLite** for lightweight/local execution or **MySQL/MariaDB** for a more traditional server-based setup, depending on deployment configuration. The user interface, if used, runs in a standard modern web browser (Chrome/Edge/Firefox), and the system should coexist with other typical developer tools (e.g., code editors/IDEs, database clients) without requiring exclusive access to system resources.

2.5 Design and Implementation Constraints

Development and implementation of the Tech Company Web Crawler and Job Openings Tracker are constrained by several practical and external factors. The system must operate in a way that is compatible with the existing project technology choices, including a Python-based backend service and a relational database layer (configured for SQLite in local execution or MySQL/MariaDB in server-style deployment). The crawler and backend must communicate over standard HTTP/HTTPS protocols and should be designed to run reliably alongside typical development tools and services on the host machine.

Because the product depends on collecting data from third-party job sources, the solution is constrained by the availability, structure, and access policies of those external websites. Page layouts may change without notice, access may be rate-limited or blocked, and certain sites may restrict automated requests. As a result,

crawling behavior must be implemented with practical safeguards such as request throttling, timeout and retry handling, and clear logging to support maintenance when sources change.

The system also faces constraints around security and responsible operation. Any stored data and any uploaded documents used for analysis (e.g., resumes) must be handled carefully to avoid accidental exposure, and the system should avoid embedding secrets directly in source code. The implementation should follow maintainable design conventions (modular crawler logic per source, clear separation between crawling, processing, storage, and API layers) so future developers can update parsers and modules as sources evolve.

2.6 User Documentation

The following user documentation components will be delivered with the Tech Company Web Crawler and Job Openings Tracker system. All documentation will be delivered electronically and written in clear, non-technical language where possible. A **User Guide/User Manual** will be provided (PDF or web-readable format) describing the main system features and step-by-step instructions for common tasks such as browsing job listings, searching and filtering jobs, viewing job details, and using interview practice and skill-gap analysis features. An **Operator/Setup Guide** will also be provided to support installation and configuration, including how to run the backend service, configure the database, trigger or schedule crawls, and interpret logs or crawler status for troubleshooting. In addition, a short **Quick Start Guide** will be included to help first-time users and operators get the system running quickly with minimal setup. If a web UI is included, basic **in-app help text** (or a short help section) may be provided to guide navigation and clarify key actions.

2.7 Assumptions and Dependencies

This section lists assumptions and external dependencies that could affect the requirements and operation of the Tech Company Web Crawler and Job Openings Tracker. If these assumptions change or dependencies become unavailable, the system may require updates to remain functional.

Assumptions

A1. It is assumed that the system will have reliable internet connectivity to access external job sources for crawling and to serve API requests to users.
A2. It is assumed that users will access the system using modern web browsers that support standard web technologies (if a web UI is provided).
A3. It is assumed that target job sources will remain reachable and will not permanently block automated access for the system's intended usage patterns.
A4. It is assumed that the database and runtime environment (local or server) will have sufficient storage and resources to retain collected job posting data and logs.

Dependencies

D1. The system depends on third-party websites (job platforms and company career pages) for job posting data; changes to page structure, access policies, authentication requirements, rate limits, or anti-bot protections may affect crawling accuracy or availability.
D2. The system depends on external libraries and frameworks (e.g., FastAPI/Uvicorn, HTTP clients, HTML parsing libraries, database drivers/ORM) remaining compatible and available.
D3. The system depends on the configured database technology (SQLite and/or MySQL/MariaDB) being available and correctly configured in the target environment.
D4. If AI-based features are enabled (e.g., interview practice, skill-gap analysis), the system depends on the availability and stability of the selected AI/LLM services or models and their APIs.

3. External Interface Requirements

3.1 User Interfaces

The system provides user interaction primarily through a **web-based user interface** (UI) and may also be accessed through **API clients** (e.g., browser, Postman, or a simple frontend page). The UI should present job listings in a clear and searchable format and allow users to view job details without requiring advanced technical knowledge. At a minimum, the user interface should support: viewing a list of job postings, searching and filtering results (e.g., by keyword, company, location, date), opening a job detail view with the full description and source link, and accessing interview/skill-gap features where available. The interface should follow common web usability conventions such as consistent navigation, readable typography, and clear actions (e.g., Search, Filter, View Details). Any error messages displayed to the user should be understandable and actionable (e.g., "No results found" or "Service unavailable, try again later"), without exposing sensitive system details such as stack traces or credentials. If administrative/operator actions are exposed through a UI (optional), they should be clearly separated from job-seeker functions and labeled to avoid accidental use (e.g., start crawl, view status, initialize data). Detailed UI layout, mockups, and screen-level design rules may be documented separately in a UI specification if needed.

3.2 Hardware Interfaces

The software has no direct dependency on specialized hardware components and does not require dedicated device integrations. It is expected to run on standard computing hardware capable of running a modern operating system and a Python runtime. Users interact with the system using standard input/output devices (keyboard, mouse/touchpad, and display) through a web browser. Network hardware (wired or wireless) is required only to provide internet connectivity for crawling external job sources and for client-server communication when the backend is deployed as a service. No additional hardware communication protocols or device-specific interfaces are required.

3.3 Software Interfaces

The system operates as a Python-based application and exposes functionality through a **REST API**. Client interfaces (such as a web UI or API testing tools) communicate with the backend over **HTTP/HTTPS** using **JSON** request and response bodies. The system stores and retrieves persistent data using a **relational database**, configured for **SQLite** in local execution or **MySQL/MariaDB** in a server-style deployment. For data collection, the crawler connects to external job sources (company career pages and job platforms) over **HTTP/HTTPS** and processes the returned content (typically HTML, and JSON where applicable) to extract job posting information. If AI-based features are enabled, the backend interfaces with internal AI modules and/or external AI services to generate interview content or perform skill-gap analysis using job descriptions and user-provided input.

3.4 Communications Interfaces

The system operates as a web-based application and communicates using standard internet protocols. Client-to-server communication occurs over **HTTP/HTTPS**, with **HTTPS** recommended for protecting data in transit.

The system is accessed through a standard web browser; no special client installation is required. Data exchanged between the client and backend is formatted as **JSON** using a request–response model. The

crawler communicates with external job sources over **HTTP/HTTPS** to retrieve job listing and detail content (typically HTML, and JSON where available).

If AI-based features are enabled, the system may communicate with AI services via **RESTful APIs**, exchanging JSON requests and responses that may include user-provided text (e.g., resume content) and job descriptions. No real-time communication is required; synchronization occurs through standard API calls and scheduled/background crawl operations.

4. System Features

4.1 Job Crawling and Data Ingestion

4.1.1 Description and Priority

Description: The system shall collect job postings from configured external sources by running crawler tasks and ingest the extracted job data into the system database for later search and use. **Priority:** High

4.1.2 Stimulus/Response Sequences

1. **Stimulus:** System Operator triggers a crawl run (manual trigger).
   Response: System starts a crawler task, attempts to fetch job source pages, and logs progress/errors.
2. **Stimulus:** Crawler successfully extracts job posting data.
   Response: System stores new job records (or updates existing ones if duplicates are detected).
3. **Stimulus:** A target source is unreachable or blocks requests.
   Response: System records a failure event/log entry and continues or stops based on configured handling.

4.1.3 Functional Requirements

- **REQ-4.1-1:** The system shall support triggering a crawler run to collect job posting data from external job sources.
- **REQ-4.1-2:** The system shall extract and store, at minimum, job title, company (or company reference), location (if available), description (if available), and source URL (or source identifier).
- **REQ-4.1-3:** The system shall avoid creating duplicate job records when the same job posting is ingested multiple times (e.g., based on source URL or other identifier).
- **REQ-4.1-4:** If crawling fails for a source due to network errors or unexpected content format, the system shall log the failure for troubleshooting.

4.2 Job Search and Viewing

4.2.1 Description and Priority

Description: The system shall allow users to retrieve job postings and apply basic filters to find relevant results. **Priority:** High

4.2.2 Stimulus/Response Sequences

1. **Stimulus:** User submits a search/filter request.
   Response: System returns a list of matching job postings.
2. **Stimulus:** User opens a job posting.
   Response: System returns the job details (and source link if available).

4.2.3 Functional Requirements

- **REQ-4.2-1:** The system shall provide an interface (UI and/or API) to list and search job postings.
- **REQ-4.2-2:** The system shall support filtering job postings by at least location and experience level when such fields exist.
- **REQ-4.2-3:** The system shall allow retrieval of job details for a selected job posting.

4.3 Interview Practice (AI-Assisted)

4.3.1 Description and Priority

Description: The system shall support interview practice sessions for users based on a selected role and optionally a specific job posting. **Priority:** Medium

4.3.2 Stimulus/Response Sequences

1. **Stimulus:** User starts an interview session for a role (and optional job).
   Response: System generates questions and returns a session identifier.
2. **Stimulus:** User submits a response.
   Response: System returns feedback and advances to the next question until completion.

4.3.3 Functional Requirements

- **REQ-4.3-1:** The system shall allow users to start an interview practice session using a role input.
- **REQ-4.3-2:** The system shall accept user responses and provide feedback per question.
- **REQ-4.3-3:** The system shall allow ending an interview session and returning a summary.

4.4 Skill Gap Analysis (Resume vs Job)

4.4.1 Description and Priority

Description: The system shall provide skill-gap analysis by comparing user-provided resume text (or file content) with a job description. **Priority:** Medium

4.4.2 Stimulus/Response Sequences

1. **Stimulus:** User submits resume text/file and selects a job.

   Response: System returns identified gaps and recommendations (if available).

4.4.3 Functional Requirements

- **REQ-4.4-1:** The system shall support skill-gap analysis given resume input and a job posting.
- **REQ-4.4-2:** The system shall return a structured analysis result (e.g., missing skills and improvement suggestions).

5. Performance and Safety

5.1 Performance Requirements

The system shall remain responsive for normal user operations while crawling and processing job data in the background. Under typical usage, job listing/search requests should return within a reasonable time (target: a few seconds), and individual job detail retrieval should be near-immediate. Crawling operations should use rate limiting and timeouts to avoid overloading external job sources and to prevent crawler activity from blocking user-facing API requests.

5.2 Safety Requirements

This product is a software-only system and does not control physical equipment. The primary safety concerns relate to preventing unintended data loss, misuse of destructive operations, and harmful system behavior during crawling.

- The system shall prevent accidental destructive operations (e.g., clearing stored job/company data) by requiring an explicit confirmation step or making such operations operator-only.
- The system shall fail safely when external sources are unavailable or return unexpected content (e.g., stop/retry gracefully and log errors rather than crashing).
- The system shall avoid generating excessive traffic to external websites by using reasonable request throttling and timeouts to reduce the risk of service disruption or blocking.

5.3 Software Quality Attributes

The following quality attributes are important for the Tech Company Web Crawler and Job Openings Tracker:

- **Usability:** The job search and viewing experience should be simple and understandable for non-technical users, with clear search/filter behavior and readable job detail pages.
- **Reliability/Robustness:** The system should handle common failure cases gracefully (e.g., unreachable sources, parsing failures, database connectivity issues) and continue operating without crashing whenever possible.
- **Maintainability:** The crawler and extraction logic should be modular so that individual sources can be updated or fixed without impacting unrelated parts of the system.
- **Testability:** Core logic (job ingestion, deduplication, and search/filter behavior) should be testable through automated tests and verifiable via API responses.
- **Portability:** The system should be able to run in typical local development environments and in a server deployment with minimal environment-specific changes.

Overall preference is for **maintainability and reliability** over highly optimized performance, since external data sources and parsing rules are expected to change over time.

5.4 Business Rules

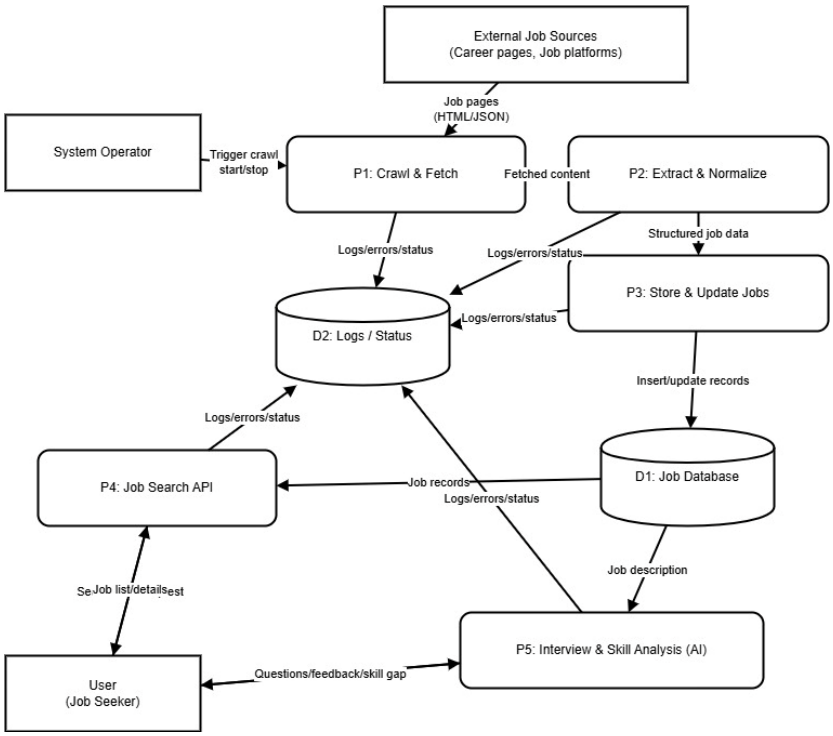
- Only trusted operators should be able to perform system operations that can modify or destroy large amounts of data (e.g., database clear/reset, initiating large crawl runs).
- The system should collect and use job posting data for tracking and user review purposes and should avoid behavior that violates external sources' stated access policies (where applicable).

Appendix A: Glossary

- **API (Application Programming Interface):** A set of endpoints that allows clients (UI or other tools) to interact with the backend system.
- **Crawler:** A software component that automatically visits web pages to collect job posting data.
- **Data Ingestion:** The process of collecting, validating, and storing extracted job data into the database.
- **Deduplication:** Identifying and preventing duplicate job records (e.g., the same posting ingested multiple times).
- **Job Source:** An external website or platform from which job postings are collected (e.g., company career pages, job boards).
- **JSON:** A structured data format used for API request/response payloads.
- **NLP (Natural Language Processing):** Techniques used to analyze and extract information from text (e.g., skills from job descriptions).
- **Skill Gap Analysis:** Comparing a resume (or resume text) against a job description to identify missing skills or areas to improve.
- **System Operator:** A maintainer/developer responsible for running crawler tasks and maintaining system health (not necessarily an in-app "admin user").
- **UI (User Interface):** The web pages/screens used by end users to search and view job postings.

Appendix B: Analysis Models

Data Flow Diagram (DFD)



Appendix C: To Be Determined List

1. **TBD-1:** Final list of external job sources to be crawled (initial set and priorities).
2. **TBD-2:** Final rule/approach for job deduplication (e.g., source URL only vs. multi-field matching).
3. **TBD-3:** Production database choice and configuration (SQLite vs MySQL/MariaDB) for deployment.
4. **TBD-4:** Whether authentication/authorization will be implemented for operator endpoints.
5. **TBD-5:** Scope and provider details for AI-based features (models, usage limits, data retention).
6. **TBD-6:** Final UI scope (API-only vs. full web UI) and any UI standards to follow.